

DOCUMENTAÇÃO TÉCNICA

Outbox Pattern Demonstração

Kotlin + Spring Boot 4 + MongoDB + Kafka — arquitetura hexagonal, domínio, camadas de resiliência e decisões técnicas.

Público-alvo: desenvolvedores/manutenção do código

Versão da aplicação: 0.0.1-SNAPSHOT

Documento gerado em: agosto de 2026

Repositório: outbox-pattern-demonstration

Sumário

1. Objetivo e escopo da POC

2. Arquitetura — Hexagonal + DDD

3. Domínio (linguagem ubíqua)

4. Camada de aplicação (usecases)

5. Infraestrutura

6. Fluxo completo, passo a passo

7. Camadas de resiliência do Outbox Pattern

8. Decisões técnicas e trade-offs

9. Referência de classes por camada

10. Lições aprendidas na implementação

11. Como rodar e testar

1. Objetivo e escopo da POC

Este projeto é uma prova de conceito (POC) que demonstra a implementação do **Outbox Pattern** — um padrão de integração que garante a publicação confiável de eventos (aqui, num tópico Kafka) mesmo diante de falhas, sem perder mensagem e sem publicar antes da transação de negócio ser confirmada.

O domínio escolhido é propositalmente simples (criação de uma **Proposta** de crédito) para que a complexidade da demonstração fique concentrada no padrão de integração em si, não nas regras de negócio.

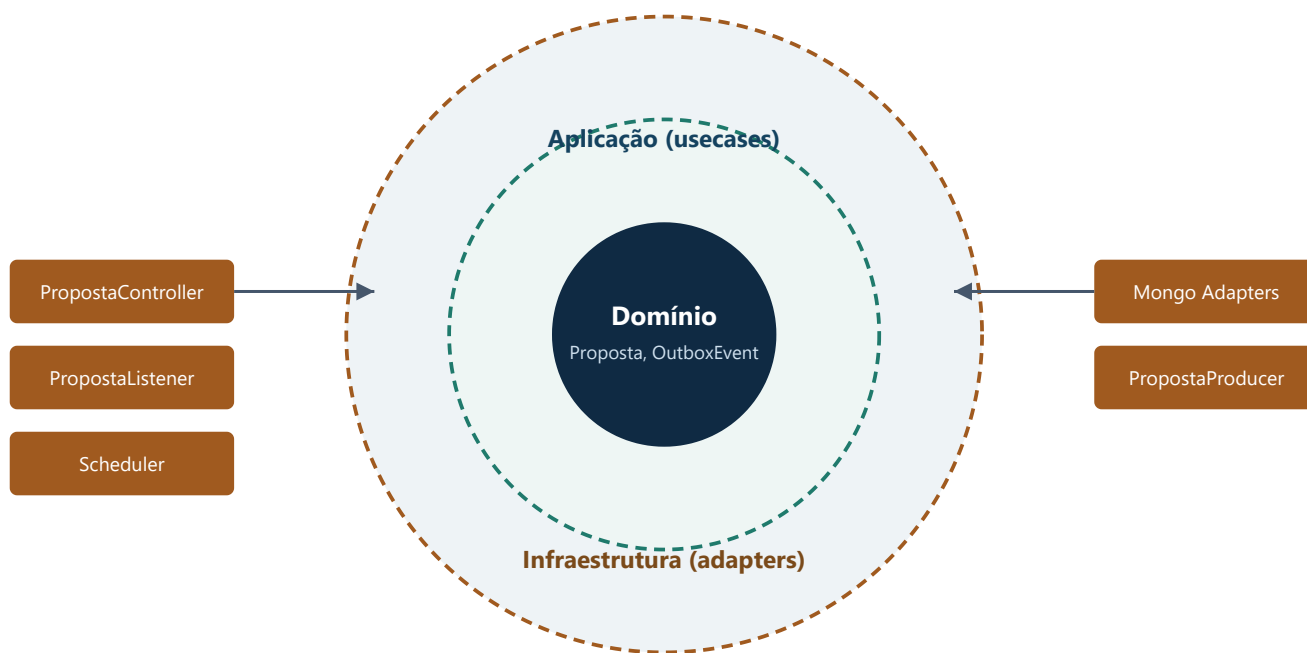
A demo sobe inteira com um único comando (`docker-compose up -d`), incluindo a própria aplicação — nenhum passo manual é necessário na apresentação.

Stack técnica

Camada	Tecnologia	Observação
Linguagem / Runtime	Kotlin + Java 21	Gradle Kotlin DSL
Framework	Spring Boot 4.1.0 (Spring Framework 7.0.8)	—
Persistência	MongoDB (Spring Data MongoDB)	Replica-set single-node — necessário para transação multi-documento (Proposta + OutboxEvent)
Mensageria	Apache Kafka (KRaft, sem Zookeeper)	Serialização Avro, sem Schema Registry
Resiliência	Resilience4j (Circuit Breaker + Retry)	No consumer Kafka
Documentação de API	springdoc-openapi (Swagger UI)	—
Observabilidade da demo	Kafka UI, Mongo Express	Camada de demonstração, não faz parte da app

2. Arquitetura — Hexagonal + DDD

O código é organizado em camadas isoladas por dependência, seguindo Arquitetura Hexagonal (Ports & Adapters) combinada com conceitos de Domain-Driven Design. A regra central: **o domínio nunca conhece infraestrutura** — nem Spring, nem Mongo, nem Kafka.



Estrutura de pacotes

Pacote	Conteúdo
<code>domain/model</code>	Proposta, StatusPropostaEnum, TipoAmortizacaoEnum, OutboxEvent, OutboxStatusEnum — Kotlin puro, sem anotação de Mongo/Spring
<code>domain/event</code>	PropostaCriadaEvent — fato de negócio imutável, payload publicado no Kafka
<code>domain/exception</code>	PropostaNaoEncontradaException
<code>domain/port/input</code>	Contratos que a aplicação oferece: CriarPropostaInputPort, ProcessarPropostaInputPort, ReprocessarOutboxInputPort
<code>domain/port/output</code>	Contratos que a infra precisa cumprir: PropostaOutputPort, OutboxOutputPort, PropostaEventPublisherOutputPort
<code>application/usecase</code>	CriarPropostaUsecase, ProcessarPropostaUsecase, ReprocessarOutboxUsecase — implementam os InputPort, orquestram domínio + OutputPort. É aqui que mora a transação.

infrastructure/input/rest	PropostaController + DTOs
infrastructure/input/kafka	PropostaListener (consumer)
infrastructure/output/persistence/mongo	Documents, Repositories (Spring Data), Adapters, Mappers
infrastructure/output/messaging/kafka	PropostaProducer (publisher) + classe Avro gerada
infrastructure/scheduler	OutboxReprocessamentoScheduler
config	MongoConfig, KafkaConfig, ResilienceConfig, OpenApiConfig

Convenção de sufixos

Sufixo	Papel
*Usecase	Caso de uso — orquestra domínio + ports, dono da transação
*InputPort	Porta de entrada (interface) — o que a aplicação oferece
*OutputPort	Porta de saída (interface) — o que a infra precisa cumprir
*Adapter	Implementação de output port na infraestrutura
*Controller / *Listener / *Producer	Adapters de entrada/saída HTTP e Kafka
*Document / *Repository / *Mapper	Persistência Mongo — nunca é o domínio
*Dto	Entrada/saída HTTP — nunca expõe o domínio direto
*Config / *Scheduler / *Exception	Configuração técnica, job agendado, exceção de domínio/aplicação

3. Domínio (linguagem ubíqua)

domain Proposta (Aggregate Root)

Representa uma proposta de financiamento em processamento assíncrono via Outbox Pattern.

Campo	Tipo	Descrição
id	String?	Nulo até ser persistida
status	StatusPropostaEnum	EM_ANDAMENTO ou PROCESSADA
tipoAmortizacao	TipoAmortizacaoEnum	SAC ou PRICE
criadaEm	Instant	Momento da criação

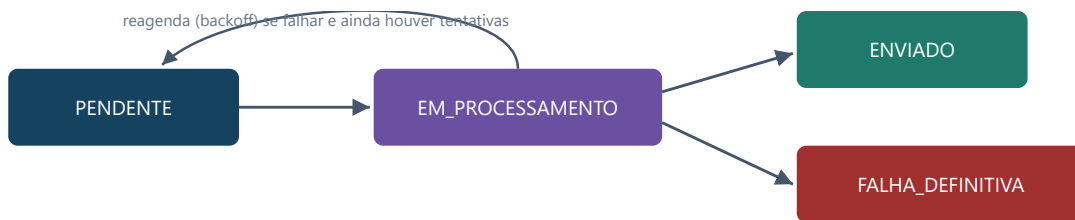
Regra de negócio: toda Proposta nasce EM_ANDAMENTO (via `Proposta.nova()`) e só transiciona para PROCESSADA quando o evento correspondente é consumido com sucesso — nunca por criação direta.

domain OutboxEvent

Registro de outbox — garante que a publicação de um evento de domínio só é tentada depois que a transação que o originou foi de fato commitada. Persistido na MESMA transação Mongo do agregado que o originou.

Campo	Tipo	Descrição
aggregateld	String	Id da Proposta associada
eventType	String	"PropostaCriada" (único tipo nesta POC)
payload	String	JSON do PropostaCriadaEvent — permite ao Scheduler republicar sem reconsultar o agregado original
status	OutboxStatusEnum	Ver ciclo de vida abaixo
tentativas	Int	Contador de tentativas de reproprocessamento
claimedBy / claimExpiraEm	String? / Instant?	Claim atômico do Scheduler — dono + expiração do lease
proximaTentativaEm	Instant?	Backoff exponencial — Scheduler ignora o registro enquanto no futuro

Ciclo de vida — OutboxStatusEnum



domain PropostaCriadaEvent (Domain Event)

Fato de negócio imutável de que uma Proposta foi criada. É exatamente o payload publicado no tópico Kafka `proposta-events` (via Avro) — carrega tudo que o consumidor precisa (`propostaId`, `status`, `tipoAmortizacao`, `criadaEm`), então o Listener nunca reconsulta a Proposta para montar sua reação.

domain PropostaNaoEncontradaException

Lançada quando o Listener recebe um evento cujo `propostaId` não existe no Mongo — cenário anômalo, diferente de uma duplicata legítima (que é idempotência tratada normalmente).

4. Camada de aplicação (usecases)

application CriarPropostaUsecase

Implementa `CriarPropostaInputPort`. É o núcleo do Outbox Pattern nesta POC:

1. `@Transactional` — grava Proposta e OutboxEvent (status PENDENTE) na mesma transação Mongo.
2. Publica um evento de aplicação em memória (Spring `ApplicationEventPublisher`) — não é o Kafka ainda.
3. `aoConfirmarCriacao`, anotado com `@TransactionalEventListener(phase = AFTER_COMMIT)`, só executa se a transação commitar — é o **fast-path**: publica no Kafka e, em caso de sucesso, marca o OutboxEvent como ENVIADO. Falha aqui não tenta de novo na hora — fica PENDENTE para o Scheduler.

application ProcessarPropostaUsecase

Implementa `ProcessarPropostaInputPort`, acionado pelo `PropostaListener`. Idempotente por construção: checa o status atual antes de transicionar — se já PROCESSADA, ignora (o Outbox Pattern só garante *at-least-once delivery*, então duplicata é esperada, não erro).

```
@CircuitBreaker(name = "processar-proposta")
@Retry(name = "processar-proposta")
override fun processar(propostaId: String) {
    val proposta = propostaOutputPort.buscarPorId(propostaId)
        ?: throw PropostaNaoEncontradaException(propostaId)
    if (proposta.status == PROCESSADA) return // idempotência
    propostaOutputPort.salvar(proposta.copy(status = PROCESSADA))
}
```

`@Retry` (interno) tenta de novo em falhas transitórias; `@CircuitBreaker` (externo) avalia o resultado da sequência inteira de tentativas e abre o circuito se as falhas persistirem.

application ReprocessarOutboxUsecase

Implementa `ReprocessarOutboxInputPort`, acionado pelo Scheduler. Único componente que reconsulta o outbox por polling. Em cada rodada (até `LOTE_MAXIMO = 50` registros):

1. Reivindica atomicamente o próximo OutboxEvent elegível (claim).
2. Desserializa o payload de volta para `PropostaCriadaEvent`.
3. Tenta republicar. Sucesso → ENVIADO.
4. Falha → incrementa tentativas; se atingiu o limite, FALHA_DEFINITIVA; senão reagenda com backoff exponencial.
5. eventType desconhecido/payload corrompido ("poison message") vai direto para FALHA_DEFINITIVA, sem gastar tentativas.

5. Infraestrutura

infra REST — PropostaController

Endpoint	Descrição
POST /api/propostas	Cria uma Proposta (body: PropostaRequestDto) → 201 com PropostaResponseDto

DTOs nunca expõem o domínio direto; a conversão é feita nos próprios objetos (`PropostaResponseDto.deDominio(...)`).

infra Persistência Mongo

Coleção	Documento	Observação
propostas	PropostaDocument	Espelho de Proposta
outbox_events	OutboxDocument	Espelho de OutboxEvent, inclui campos de claim/backoff

`OutboxRepositoryAdapter.reivindicarProximoPendente` usa `MongoTemplate.findAndModify` (atômico) — mesmo com duas instâncias da app rodando a mesma query ao mesmo tempo, só uma "vence" o claim de um dado documento.

infra Mensageria Kafka + Avro

Componente	Papel
PropostaProducer	Implementa <code>PropostaEventPublisherOutputPort</code> . Serializa Avro binário manualmente (sem header do Confluent Schema Registry), publica em <code>proposta-events</code> usando <code>proposald</code> como chave de partição. Aguarda confirmação com timeout de 5s.
PropostaListener	Consumer group isolado <code>proposta-listener</code> . Desserializa o mesmo Avro binário e aciona <code>ProcessarPropostaInputPort</code> .
Schema Avro	<code>src/main/avro/proposta_criada_event.avsc</code> — versionado no repositório, gerado via plugin Gradle Avro

infra OutboxReprocessamentoScheduler

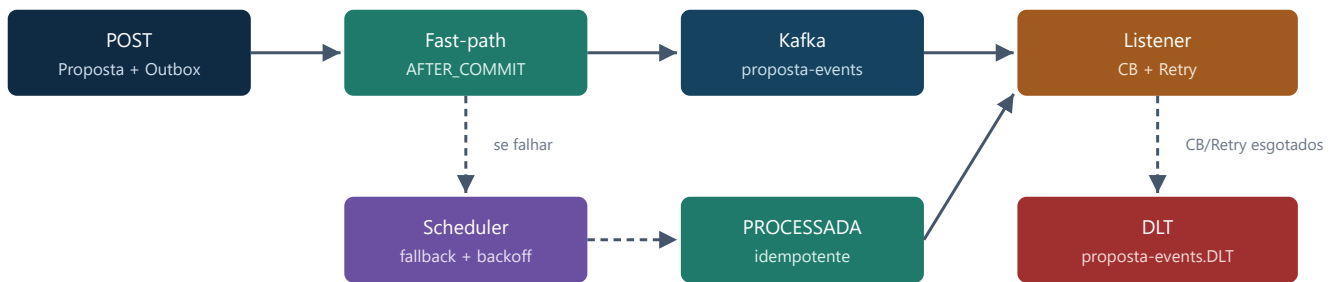
`@Scheduled(fixedDelayString = "${outbox.scheduler.fixed-delay-ms:5000})`. Opt-in via `outbox.scheduler.enabled` — desabilitar não quebra o fluxo síncrono. Gera um id de instância aleatório (`dono`) por startup, usado no claim atômico.

config Classes de configuração

Classe	Responsabilidade
MongoConfig	Registra o <code>MongoTransactionManager</code> — sem ele, <code>@Transactional</code> não teria efeito
KafkaConfig	<code>@EnableKafka</code> ; producer/consumer factories (bytes crus); <code>DefaultErrorHandler</code> + <code>DeadLetterPublishingRecoverer</code> apontando explicitamente para <code>proposta-events.DLT</code>
ResilienceConfig	Loga toda tentativa de retry e transição de estado do Circuit Breaker (observabilidade da demo)
OpenApiConfig	Metadados do Swagger UI

6. Fluxo completo, passo a passo

1. **POST /api/propostas** — cria Proposta (EM_ANDAMENTO) + OutboxEvent (PENDENTE) na mesma transação Mongo.
2. **Fast-path** — após o commit, publica imediatamente no Kafka (Avro, chave = propostald). Sucesso → OutboxEvent vira ENVIADO.
3. **Fallback (Scheduler)** — cobre o que o fast-path não confirmou (crash, Kafka fora do ar): claim atômico dos PENDENTE elegíveis, republica com backoff exponencial; após o limite de tentativas, FALHA_DEFINITIVA.
4. **Consumo** — PropostaListener consome o tópico, aciona ProcessarPropostaUsecase com Circuit Breaker + Retry.
5. **Idempotência** — se a Proposta já está PROCESSADA, a mensagem é ignorada (no-op) — esperado sob at-least-once delivery.
6. **Falha definitiva de processamento** — Circuit Breaker/Retry esgotados → mensagem original vai para a DLT (`proposta-events.DLT`), visível no Kafka UI.



7. Camadas de resiliência

Camada	Cobre	Mecanismo
Fast-path pós-commit	Caminho feliz — latência mínima	@TransactionalEventListener(AFTER_COMMIT)
Scheduler de fallback	Kafka fora do ar no momento da criação, crash entre commit e publish	Claim atômico (findAndModify, dono+expiração) + backoff exponencial (5s → 300s) + limite de tentativas (default 5) → FALHA_DEFINITIVA
Circuit Breaker + Retry (consumo)	Falha transitória ao processar (ex.: Mongo instável)	Resilience4j — 3 tentativas, circuito abre com ≥50% de falha numa janela de 10 chamadas (mínimo 5)
Dead Letter Topic	Falha definitiva de processamento (CB/Retry exauridos) ou mensagem corrompida	DefaultErrorHandler + DeadLetterPublishingRecoverer → proposta-events.DLT

Nota de design: o Resilience4j faz o retry *dentro* do usecase; o container Kafka é configurado com `FixedBackOff(0, 0)` — não tenta de novo por conta própria, evitando duplicar a lógica de retry em duas camadas.

8. Decisões técnicas e trade-offs

Polling Publisher vs. CDC (Change Data Capture)

	Polling Publisher (usado nesta POC)	CDC (Debezium / Change Streams)
Como funciona	Processo consulta a coleção outbox periodicamente	Lê o log de transação do banco
Latência	Segundos (configurável)	Milissegundos
Infraestrutura extra	Nenhuma	Kafka Connect + operação de um connector
Quando usar	Protótipos, POCs, clareza didática	Produção em alta escala

Decisão consciente para esta POC: Polling (fast-path pós-commit + Scheduler de fallback), sem Debezium/Change Streams — infraestrutura extra não se paga para uma demonstração; o objetivo é clareza didática do padrão, não throughput de produção.

At-least-once + idempotência (não exactly-once)

O Outbox Pattern garante que a mensagem *será* entregue, mas não garante entrega única. A escolha de projeto foi assumir isso explicitamente: o consumidor **precisa** ser idempotente (checar o status atual antes de agir), em vez de tentar (inutilmente) garantir exactly-once na infraestrutura.

Sem Schema Registry

O schema Avro é versionado no próprio repositório (`src/main/avro`), sem Confluent Schema Registry. Trade-off aceitável numa POC com um único produtor/consumidor e um único tipo de evento — não escalaria bem com múltiplos times/schemas evoluindo independentemente.

MongoDB único (não DocumentDB)

O container `mongo` real (replica-set single-node) representa o Amazon DocumentDB (wire-compatible) sem custo — DocumentDB só é suportado na imagem PRO do LocalStack, incompatível com o objetivo de demo 100% gratuita.

Recursos AWS/LocalStack removidos

Estava planejado usar LocalStack para simular AWS Secrets Manager. A decisão final foi remover completamente essa camada: nenhuma credencial sensível real precisa ser gerenciada nesta POC, e a complexidade extra não se justificava para o objetivo da demonstração.

9. Referência de classes por camada

Classe	Camada	Papel
Proposta	domain/model	Aggregate Root
StatusPropostaEnum	domain/model	EM_ANDAMENTO / PROCESSADA
TipoAmortizacaoEnum	domain/model	SAC / PRICE
OutboxEvent	domain/model	Registro de outbox
OutboxStatusEnum	domain/model	PENDENTE / EM_PROCESSAMENTO / ENVIADO / FALHA_DEFINITIVA
PropostaCriadaEvent	domain/event	Domain Event / payload Kafka
PropostaNaoEncontradaException	domain/exception	Exceção de aplicação
CriarPropostaInputPort / ProcessarPropostaInputPort / ReprocessarOutboxInputPort	domain/port/input	Contratos de entrada
PropostaOutputPort / OutboxOutputPort / PropostaEventPublisherOutputPort	domain/port/output	Contratos de saída
CriarPropostaUsecase	application/usecase	Cria Proposta + Outbox, fast-path
ProcessarPropostaUsecase	application/usecase	Consome evento, idempotente, CB+Retry
ReprocessarOutboxUsecase	application/usecase	Fallback do Scheduler
PropostaController	infrastructure/input/rest	POST /api/propostas
PropostaListener	infrastructure/input/kafka	Consumer Kafka
PropostaDocument / PropostaMongoRepository / PropostaRepositoryAdapter / PropostaMapper	infrastructure/output/persistence/mongo	Persistência da Proposta
OutboxDocument / OutboxMongoRepository / OutboxRepositoryAdapter / OutboxMapper	infrastructure/output/persistence/mongo	Persistência do outbox + claim atômico

PropostaProducer	infrastructure/output/messaging/kafka	Publisher Avro
OutboxReprocessamentoScheduler	infrastructure/scheduler	Job agendado
MongoConfig / KafkaConfig / ResilienceConfig / OpenApiConfig	config	Configuração técnica

10. Lições aprendidas na implementação

Problemas reais encontrados rodando a aplicação de ponta a ponta — nenhum deles aparecia como erro de compilação ou exceção no boot, o que reforça a importância de validar cenários de falha na prática, não só ler o código.

Spring Boot 4.1 usa Jackson 3 por padrão

O `ObjectMapper` autoconfigurado é `tools.jackson.databind.ObjectMapper`, não `com.fasterxml.jackson.databind.ObjectMapper` — o pacote antigo ainda compila (está no classpath transitivamente), mas a injeção falha silenciosamente no boot.

CRLF quebrava scripts dentro dos containers Linux

`core.autocrlf=true` no Windows reescrevia `gradlew` e os scripts de init a cada checkout, quebrando o shebang. Corrigido com `.gitattributes` forçando `eol=lf`.

Fast-path e Scheduler duplicavam publicação

O Scheduler considerava qualquer PENDENTE elegível imediatamente, competindo com o fast-path em quase toda criação. Corrigido com um período de carência antes de um PENDENTE "de primeira tentativa" ficar elegível.

PropostaProducer era fire-and-forget

`kafkaTemplate.send()` só lançava exceção de forma assíncrona (futuro nunca inspecionado) — o fast-path/Scheduler nunca veriam uma falha real do Kafka. Corrigido bloqueando em `.get(timeout)`.

@EnableKafka ausente — Listener nunca era processado

Sem erro, sem warning: o consumer simplesmente não inicializava. Confirmado só ao notar que nenhum consumer group aparecia no Kafka.

DeadLetterPublishingRecoverer — sufixo de tópico divergente

O default do Spring Kafka 4.1 é `<tópico>-dlr` (hífen), não `.DLT` — publicava num tópico diferente do já provisionado. Corrigido com um destination resolver explícito.

@CircuitBreaker/@Retry ignorados silenciosamente

Faltava `aspectjweaver` no classpath. Spring Boot 4 não tem mais um `spring-boot-starter-aop` dedicado (módulos desmembrados).

Timeout do Mongo (30s default) tornava falhas lentas de diagnosticar

Reduzido para 5s via `serverSelectionTimeoutMS` na URI — mesmo raciocínio do `MAX_BLOCK_MS_CONFIG` do producer Kafka.

11. Como rodar e testar

Pré-requisito único: Docker Desktop.

```
docker-compose up -d
```

Sobe Mongo (replica-set), Kafka (KRaft), a aplicação, Swagger UI, Kafka UI e Mongo Express — sem nenhum passo manual adicional. Instruções completas de demonstração (incluindo os cenários de falha) estão no `README.md` do repositório.

Serviço	URL
Swagger UI	http://localhost:8080/swagger-ui.html
Kafka UI	http://localhost:8082
Mongo Express	http://localhost:8081